Nama : Eka Ahmad Ferdinand Mahtrus

PROGRESS III

KONTROL MOTOR MENGGUNAKAN PYTHON

3.1. Tujuan Modul

- Merancang dan mengimplementasikan aplikasi antarmuka pengguna grafis (*Graphical User Interface* / GUI) mandiri berbasis desktop menggunakan bahasa pemrograman Python dan pustaka Tkinter.
- Membangun protokol komunikasi serial secara lokal antara komputer (sebagai *master*) dan mikrokontroler ESP32 (sebagai *slave*) memanfaatkan pustaka PySerial.
- Menguji dan mengevaluasi keandalan mekanisme *string parsing* dan sinkronisasi data menggunakan karakter terminasi (*newline*) dalam mengendalikan aktuator motor DC secara *real-time* tanpa ketergantungan pada jaringan internet.

3.2. Skema Wiring (Rangkaian)

Topologi pengkabelan pada tahap ini difokuskan pada komunikasi kabel fisik, mempertahankan konfigurasi *driver* motor yang identik dengan tahap sebelumnya, dengan detail sebagai berikut:

- **Jalur Komunikasi Utama:** Komputer atau laptop dihubungkan secara langsung ke *port* Micro-USB/Type-C ESP32 menggunakan kabel data untuk memfasilitasi transmisi antarmuka UART (*Universal Asynchronous Receiver-Transmitter*).
- **Pin 27 (ESP32)** dihubungkan ke pin **IN1** *driver* motor L298N (kendali arah).
- **Pin 26 (ESP32)** dihubungkan ke pin **IN2** *driver* motor L298N (kendali arah).
- **Pin 12 (ESP32)** dihubungkan ke pin **ENA** *driver* motor L298N (penerima sinyal kontrol kecepatan PWM berfrekuensi 30 kHz).

3.3. Konsep & Logika Sistem

Sistem beroperasi berdasarkan arsitektur *Master-Slave* secara lokal. Aplikasi Python di komputer bertindak sebagai *Master* yang menyediakan panel kendali interaktif berupa

tombol operasi dan *slider* parameter kecepatan. Setiap kali pengguna melakukan interaksi pada antarmuka GUI, program Python akan merakit perintah tersebut menjadi teks berformat ASCII (contoh: "1", "0", atau "PWM:200"), menyisipkan karakter terminasi baris baru (\n), mengonversinya menjadi *byte*, lalu mengirimkannya ke saluran COM Port USB.

Di sisi lain, ESP32 bertindak sebagai *Slave* yang secara pasif terus memantau (*listening*) ketersediaan data di jalur Serial pada kecepatan 115200 *baud*. Ketika paket data terdeteksi, mikrokontroler tidak langsung mengeksekusinya keping demi keping, melainkan menyimpannya sementara ke dalam *buffer* memori hingga menemukan karakter terminasi \n. Setelah pesan utuh diterima, mikrokontroler akan mengekstrak makna dari teks tersebut dan menerjemahkannya menjadi lonjakan arus fisik ke *driver* motor.

3.4. Daftar Perintah pada Program

A. Sisi Perangkat Lunak (Python GUI)

- Program aplikasi desktop dibangun secara modular menggunakan paradigma *Object-Oriented Programming* (OOP) dalam kelas MotorControllerApp. Antarmuka disusun menggunakan pustaka tkinter yang dibagi menjadi beberapa bingkai fungsional: pendeteksi COM Port dinamis, tombol sakelar biner, dan instrumen *slider* rentang 0-255.
- Logika transmisi dienkapsulasi dalam fungsi send_command(cmd). Fungsi ini bertugas memastikan bahwa koneksi Serial terbuka sebelum melakukan injeksi teks perintah. Sistem secara eksplisit menggabungkan perintah pengguna dengan *newline* melalui sintaks self.serial_port.write((cmd + '\n').encode('utf-8')). Penambahan *newline* ini bersifat esensial untuk mencegah tabrakan data (*data collision*) apabila pengguna menggeser *slider* kecepatan dengan ritme yang sangat cepat.

B. Sisi Perangkat Keras (Firmware ESP32)

- Program mikrokontroler dirancang khusus untuk membedah data teks masuk (*string parsing*). Di dalam perulangan utama loop(), mikrokontroler menanti data dengan instruksi Serial.available() > 0. Ekstraksi data dilakukan secara presisi melalui Serial.readStringUntil('\n'), yang secara otomatis memisahkan satu perintah penuh dari aliran data yang panjang.

- Setelah perintah ditangkap, program melakukan pembersihan spasi menggunakan command.trim(). Sistem kemudian menerapkan percabangan kondisional (if - else if) untuk mengidentifikasi jenis instruksi. Apabila instruksi bersifat absolut ("1" atau "0"), sistem seketika mengubah status tegangan pin digital. Namun, apabila perintah diawali dengan prefiks spesifik (dideteksi melalui command.startsWith("PWM:")), program akan memotong *string* tersebut (substring(4)), mengonversi nilai teks yang tersisa menjadi variabel integer matematis (toInt()), dan mengeksekusinya sebagai besaran rasio tugas (*duty cycle*) pada fungsi ledcWrite().

3.5. Source Code & Penjelasan Logika Program

- **Source code Python (Tkinter GUI)**

```
"""
================================================================
============================

Project: Python GUI untuk Kontrol Motor ESP32 via Serial

Description: Aplikasi desktop berbasis Tkinter untuk mengendalikan arah dan
kecepatan

        motor DC yang terhubung ke ESP32 melalui komunikasi Serial (Kabel
USB).

        Program ini bertindak sebagai pengganti Serial Monitor Arduino IDE.



Prasyarat:

Pastikan Anda telah menginstal library pyserial sebelum menjalankan kode ini.

Cara install: pip install pyserial
```

```
================================================================
==============================
"""



import tkinter as tk

from tkinter import ttk, messagebox

import serial

import serial.tools.list_ports



class MotorControllerApp:

    def __init__(self, root):

        self.root = root

        # Pengaturan jendela utama (Window)

        self.root.title("Python ESP32 Motor Controller")

        self.root.geometry("400x380")

        self.root.resizable(False, False) # Mencegah ukuran jendela diubah



        # Variabel untuk menyimpan status port serial

        self.serial_port = None
```

```
        # Inisialisasi antarmuka pengguna (GUI) dan pemindaian port
        self.create_widgets()
        self.refresh_ports()

    def create_widgets(self):
        """Membangun elemen-elemen visual (Widget) pada jendela aplikasi."""

        # ==========================================================
        # 1. FRAME KONEKSI SERIAL
        # ==========================================================
        conn_frame = ttk.LabelFrame(self.root, text="Pengaturan Koneksi Serial")
        conn_frame.pack(pady=10, padx=10, fill="x")

        # Dropdown (Combobox) untuk memilih Port COM
        self.port_var = tk.StringVar()
        self.port_cb = ttk.Combobox(conn_frame, textvariable=self.port_var,
state="readonly", width=15)
        self.port_cb.grid(row=0, column=0, padx=10, pady=10)
```

```
        # Tombol untuk memindai ulang perangkat USB yang terhubung
        btn_refresh = ttk.Button(conn_frame, text="Refresh Port",
command=self.refresh_ports)
        btn_refresh.grid(row=0, column=1, padx=5, pady=10)

        # Tombol untuk menyambung/memutus koneksi ke ESP32
        self.btn_connect = ttk.Button(conn_frame, text="Connect",
command=self.toggle_connection)
        self.btn_connect.grid(row=0, column=2, padx=5, pady=10)

        #
==========================================================
=
        # 2. FRAME KONTROL MOTOR DASAR (ON/OFF)
        #
==========================================================
=
        ctrl_frame = ttk.LabelFrame(self.root, text="Kontrol Motor Dasar")
        ctrl_frame.pack(pady=10, padx=10, fill="x")

        # Tombol MOTOR ON (Mengirimkan karakter "1" ke ESP32)
```

```
        btn_on = tk.Button(ctrl_frame, text="MOTOR ON", bg="#4CAF50",
fg="white", font=("Arial", 10, "bold"), command=lambda:
self.send_command("1"))

        btn_on.pack(side="left", expand=True, fill="x", padx=10, pady=15)



        # Tombol MOTOR OFF (Mengirimkan karakter "0" ke ESP32)

        btn_off = tk.Button(ctrl_frame, text="MOTOR OFF", bg="#F44336",
fg="white", font=("Arial", 10, "bold"), command=lambda:
self.send_command("0"))

        btn_off.pack(side="right", expand=True, fill="x", padx=10, pady=15)



        #
==========================================================
=

        # 3. FRAME KONTROL KECEPATAN (PWM)

        #
==========================================================
=

        pwm_frame = ttk.LabelFrame(self.root, text="Kontrol Kecepatan (PWM)")

        pwm_frame.pack(pady=10, padx=10, fill="x")



        # Slider (Scale) untuk memilih nilai kecepatan antara 0 hingga 255
```

```
        self.pwm_slider = tk.Scale(pwm_frame, from_=0, to=255,
orient="horizontal", tickinterval=51)

        self.pwm_slider.set(150) # Set nilai default awal ke 150

        self.pwm_slider.pack(fill="x", padx=10, pady=5)



        # Tombol untuk mengeksekusi pengiriman nilai slider ke ESP32

        btn_set_pwm = ttk.Button(pwm_frame, text="Set Kecepatan",
command=self.set_pwm)

        btn_set_pwm.pack(pady=10)



        #
==========================================================
=

        # 4. LABEL STATUS

        #
==========================================================
=

        # Menampilkan indikator visual apakah aplikasi sedang terhubung ke ESP32
atau tidak

        self.status_label = ttk.Label(self.root, text="Status: Disconnected",
foreground="red", font=("Arial", 10, "bold"))

        self.status_label.pack(side="bottom", pady=10)
```

```
    # --- FUNGSI-FUNGSI LOGIKA SISTEM ---



    def refresh_ports(self):

        """Mencari semua Port COM (Kabel USB) yang tersedia di laptop/PC."""

        ports = serial.tools.list_ports.comports()

        # Masukkan daftar port yang ditemukan ke dalam dropdown

        self.port_cb['values'] = [port.device for port in ports]

        # Jika ada port yang terdeteksi, otomatis pilih port pertama

        if self.port_cb['values']:

            self.port_cb.current(0)



    def toggle_connection(self):

        """Menangani logika menghubungkan (Connect) atau memutus (Disconnect) komunikasi Serial."""

        # Jika port sedang terbuka (sedang konek), maka putuskan koneksinya

        if self.serial_port and self.serial_port.is_open:

            self.serial_port.close()

            # Kembalikan tampilan GUI ke mode awal

            self.btn_connect.config(text="Connect")

            self.status_label.config(text="Status: Disconnected", foreground="red")
```

```
            self.port_cb.config(state="readonly")

        else:

            # Jika sedang tidak konek, ambil port dari dropdown dan coba hubungkan

            port = self.port_var.get()

            if port:

                try:

                    # Buka koneksi serial. Baudrate 115200 WAJIB SAMA dengan
Serial.begin(115200) di ESP32

                    self.serial_port = serial.Serial(port, 115200, timeout=1)



                    # Ubah tampilan GUI untuk menunjukkan keberhasilan koneksi

                    self.btn_connect.config(text="Disconnect")

                    self.status_label.config(text=f"Status: Connected to {port}",
foreground="green")

                    self.port_cb.config(state="disabled") # Kunci dropdown agar port tidak
diubah saat konek

                except Exception as e:

                    # Tampilkan pesan error jika gagal (biasanya karena port sedang
dipakai Arduino IDE)

                    messagebox.showerror("Error", f"Gagal terhubung ke {port}:
{str(e)}\n\nPastikan Serial Monitor di Arduino IDE sudah ditutup!")

            else:
```

```
            # Peringatan jika pengguna menekan Connect tapi belum memilih port
            messagebox.showwarning("Peringatan", "Pilih port COM terlebih dahulu!")

    def send_command(self, cmd):
        """Mengirimkan string perintah ke ESP32 melalui jalur Serial."""
        # Pastikan koneksi sudah terbuka sebelum mengirim data
        if self.serial_port and self.serial_port.is_open:
            try:
                # Tambahkan karakter newline ('\n') di akhir teks.
                # Ini sangat krusial karena kode ESP32 menggunakan logika (c == '\n') untuk mendeteksi akhir perintah.
                self.serial_port.write((cmd + '\n').encode('utf-8'))
            except Exception as e:
                messagebox.showerror("Error", f"Gagal mengirim data: {str(e)}")
        else:
            messagebox.showwarning("Peringatan", "Silakan klik tombol 'Connect' terlebih dahulu!")

    def set_pwm(self):
        """Menyusun format perintah PWM untuk dikirim ke ESP32."""
```

```
        # Ambil angka aktual dari slider
        val = self.pwm_slider.get()
        # Gabungkan teks "PWM:" dengan angka, lalu kirim ke fungsi send_command
        # Contoh output akhir yang dikirim: "PWM:150\n"
        self.send_command(f"PWM:{val}")


# Blok utama untuk menjalankan aplikasi
if __name__ == "__main__":
    root = tk.Tk()
    app = MotorControllerApp(root)
    root.mainloop()
```

- **Source code C++ (ESP32 Receiver)**

```
/*
 *
============================================================
============================
 * Project: ESP32 Motor Controller via Python GUI (Serial Communication)
 * Description: Kode ini dirancang khusus untuk menerima dan mengeksekusi
perintah
 *              yang dikirimkan dari aplikasi desktop Python (Tkinter/PySerial).
```

```
 *            Mikrokontroler bertindak sebagai 'slave' yang mengeksekusi instruksi

 *            melalui komunikasi Serial (Kabel USB).

 *

 * Kompatibilitas: Menggunakan sintaks PWM ESP32 Core versi 3.x ke atas.

 *
================================================================
==============================

 */



//
================================================================
=

// 1. KONFIGURASI PIN HARDWARE

//
================================================================
=

const int motor1Pin1 = 27;  // Pin IN1 pada driver motor (Kendali arah putaran)

const int motor1Pin2 = 26;  // Pin IN2 pada driver motor (Kendali arah putaran)

const int enable1Pin = 12;  // Pin ENA pada driver motor (Kendali kecepatan via
PWM)
```

```
//
==========================================================
=

// 2. PARAMETER PWM (Pulse Width Modulation)

//
==========================================================
=

const int freq = 30000;     // Frekuensi 30 kHz (tinggi agar menghilangkan suara
dengung pada motor)

const int resolution = 8;   // Resolusi 8-bit (Rentang kecepatan dari 0 hingga 255)



//
==========================================================
=

// 3. SETUP AWAL (Inisialisasi Sistem)

//
==========================================================
=

void setup() {

  // Memulai jalur komunikasi Serial dengan baudrate 115200

  // (Wajib selaras dengan konfigurasi di script Python: serial.Serial(port, 115200))

  Serial.begin(115200);
```

```
  // Mengatur pin-pin sebagai keluaran (OUTPUT)
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);

  // Menghubungkan pin Enable dengan generator PWM bawaan ESP32 (Sintaks Core 3.x)
  ledcAttach(enable1Pin, freq, resolution);

  // --- PROSEDUR KEAMANAN (SAFETY BOOT) ---
  // Pastikan motor dalam keadaan mati (Brake) saat mikrokontroler baru saja menyala
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(enable1Pin, 0);

  // Mengirimkan pesan status ke Serial Monitor / Terminal Python
  Serial.println("[SYSTEM INFO] Sistem Siap! Menunggu perintah dari Python GUI...");
}
```

```
//
============================================================
=

// 4. LOOP UTAMA (Pendengar Instruksi Serial)

//
============================================================
=

void loop() {

  // Mengecek apakah ada paket data (byte) yang dikirimkan dari komputer (Python)

  if (Serial.available() > 0) {



    // Membaca seluruh rentetan karakter hingga menemukan tanda Enter / Newline
('\n').

    // Karakter '\n' inilah yang disisipkan oleh fungsi send_command() di kode
Python.

    String command = Serial.readStringUntil('\n');



    // Membersihkan spasi kosong (whitespace) atau karakter tersembunyi (\r) di
ujung teks

    command.trim();



    // ----------------------------------------------------
```

```
  // CABANG LOGIKA EKSEKUSI PERINTAH
  // -------------------------------------------------------


  // A. Perintah Menghidupkan Motor
  if (command == "1") {
    digitalWrite(motor1Pin1, HIGH); // Mengatur arah arus (Maju)
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, 200);     // Memberikan nilai PWM 200 (sebagai torsi awal)

    Serial.println(">>> [AKSI] Motor Dinyalakan");
  }

  // B. Perintah Mematikan Motor
  else if (command == "0") {
    digitalWrite(motor1Pin1, LOW);  // Memutus arah arus
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, 0);       // Menghentikan putaran sepenuhnya (PWM 0)

    Serial.println(">>> [AKSI] Motor Dimatikan");
```

```
    }


    // C. Perintah Mengubah Kecepatan dari Slider (Contoh Format: "PWM:150")
    else if (command.startsWith("PWM:")) {
      // Memotong string untuk mengambil karakter ke-4 dan seterusnya (hanya menyisakan angkanya)
      String pwmValueStr = command.substring(4);


      // Mengonversi string angka (misal "150") menjadi tipe data integer matematis
      int pwmValue = pwmValueStr.toInt();


      // Menerapkan nilai kecepatan tersebut ke perangkat keras (driver motor)
      ledcWrite(enable1Pin, pwmValue);

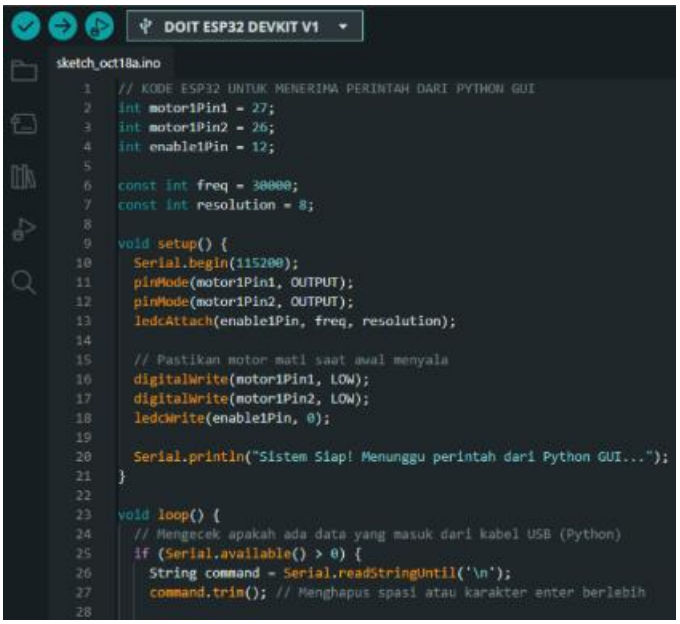

      // Mengirimkan umpan balik (feedback log) ke terminal
      Serial.print(">>> [AKSI] Kecepatan (PWM) diubah menjadi: ");
      Serial.println(pwmValue);
    }
  }
}
```

- **Penjelasan Program**

Program pada sisi mikrokontroler ESP32 difokuskan pada penerimaan data Serial dan pembangkitan sinyal kendali fisik. Pada blok setup(), mikrokontroler menginisialisasi jalur komunikasi UART melalui instruksi Serial.begin(115200). Selanjutnya, pin keluaran dikonfigurasi menggunakan standar pustaka ESP32 Core versi 3.x, di mana pin 12 (ENA) ditautkan ke generator *Pulse Width Modulation* (PWM) berfrekuensi 30 kHz dan beresolusi 8-bit melalui fungsi ledcAttach(). Sistem juga menerapkan rutinitas *safety boot* dengan memaksa nilai awal PWM ke angka 0 agar motor tidak berputar secara liar saat perangkat baru dihidupkan.

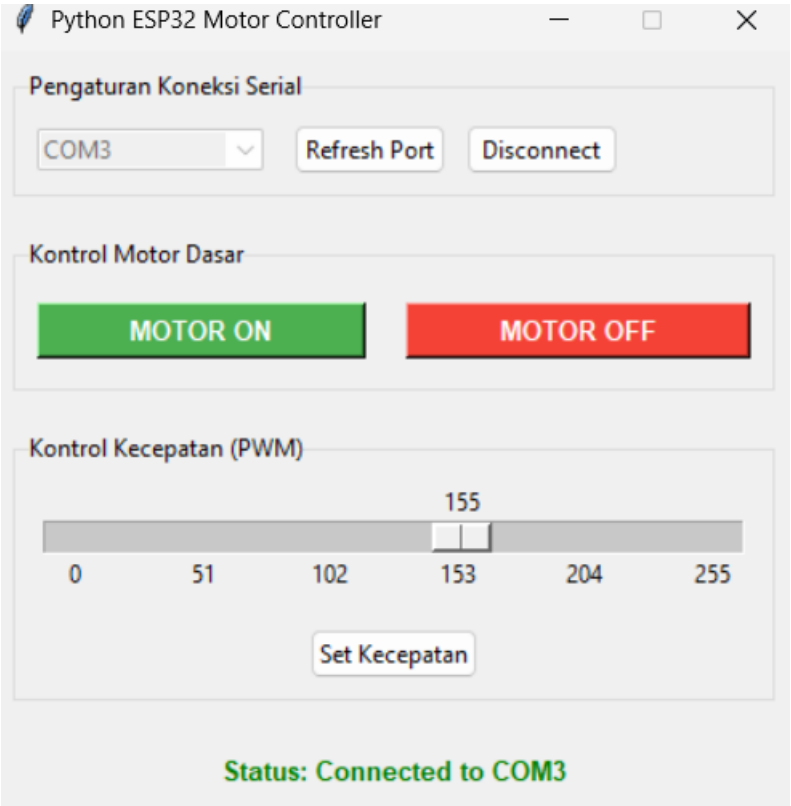
Pada blok loop(), mikrokontroler beroperasi secara pasif menanti masuknya interupsi data fisik menggunakan kondisi Serial.available() > 0. Ekstraksi data dilakukan menggunakan metode pembacaan berbasis terminasi, yakni Serial.readStringUntil('\n'). Fungsi ini akan menahan proses pembacaan di dalam *buffer* sementara hingga mendeteksi karakter *newline* yang dikirimkan oleh program Python. Setelah satu kalimat instruksi utuh tertangkap, fungsi trim() dieksekusi untuk memurnikan teks dari sisa spasi atau karakter tersembunyi (*carriage return*).

3.6. Hasil Pengujian dan Analisa



Gambar 1. Proses Upload

- Sebelum pengujian GUI yang ada di Python, perangkat lunak yang berbasis C++ harus dikompilasi dan diunggah terlebih dahulu ke mikrokontroler ESP32 melalui Arduino IDE



- Lalu jika sudah, GUI yang di Python dan jika sudah sesuai maka akan ada notifikasi ”Status : Connected to COM3” apabila langkah langkahnya sudah sesuai

- **Analisa Keseluruhan**

Berdasarkan implementasi dan pengujian yang telah dilakukan, integrasi sistem kendali motor melalui antarmuka GUI Python terbukti memberikan tingkat responsivitas operasional yang jauh lebih tinggi dan deterministik dibandingkan sistem nirkabel (IoT). Transmisi data melalui kabel fisik USB berhasil mengeliminasi potensi latensi jaringan, fluktuasi *ping*, dan risiko *packet loss* yang sering menjadi kendala utama pada platform berbasis *cloud*.

Penerapan metode penyisipan karakter terminasi (\n) dari aplikasi Python yang dipasangkan dengan fungsi pembacaan readStringUntil('\n') di ESP32 terbukti

merupakan mekanisme sinkronisasi yang sangat tangguh. Protokol sederhana ini memastikan mikrokontroler tidak pernah mengeksekusi instruksi yang terpotong atau cacat (*corrupted data*), bahkan ketika dibanjiri oleh ratusan baris perintah kecepatan dalam waktu singkat saat pengguna memanipulasi *slider* GUI secara agresif. Secara teknis, Modul Progress III ini berhasil mendemonstrasikan sebuah infrastruktur stasiun kendali lokal (*Local Control Station*) yang mandiri, aman dari gangguan sinyal eksternal, dan sangat ideal untuk diaplikasikan pada lingkungan tertutup yang tidak memiliki akses internet.